ATOMIZED ROADMAP

STM32 Bare-Metal & RTOS (Fully Itemized)

LEVEL 1 — BITWISE OPERATIONS AND NATIVE MEMORY CONTROL

001. Manually calculate a binary and hexadecimal mask to isolate a specific bit.
002. Apply the logical AND (&) operation to clear (set to 0) a register bit without altering the others.
003. Apply the logical OR (|) operation to activate (set to 1) a configuration bit.
004. Combine the NOT (~) operator with AND (&) to safely turn off a specific bit.
005. Apply the XOR (^) operator to accurately toggle a status bit.
006. Use logical left shifts (<<) to position a multi-bit value within a register field.
007. Use logical right shifts (>>) to extract and read a multi-bit field from a register.
008. Design a compound mask with shifts to clear and then write a multi-bit configuration field.
009. Implement a synchronous polling read by analyzing a status flag using an AND mask.
010. Declare a C structure using the '__attribute__((packed))' attribute to map memory precisely.
011. Define absolute physical addresses of base registers using C macros and pointers.
012. Explain and apply the 'volatile' qualifier to pointers to prevent the compiler optimizer from removing hardware reads.
013. Create a custom full register map using 'struct' and volatile pointers for a simulated hardware block.
014. Implement a shared global variable modified within an ISR (Interrupt Service Routine) properly qualified as 'volatile'.
015. Use 'uint32_t*' opaque pointers to write data directly into physical addresses of the ARM memory map.

LEVEL 2 — BASE SILICON CONFIGURATION (RCC AND GPIO BARE-METAL)

016. Locate the base addresses of the RCC (Reset and Clock Control) block in the Reference Manual.

017. Write to the corresponding RCC register to enable the clock for the selected GPIO peripheral.

018. Identify the AHB bus clock prescaler configuration registers in the manual.

019. Modify the RCC prescaler register bits to manually configure the AHB bus speed.

020. Locate and configure the APB1 and APB2 bus prescaler bits using clock control registers.

021. Map the addresses of the GPIOx_MODER, GPIOx_OTYPER, GPIOx_PUPDR, and GPIOx_OSPEEDR registers of the microcontroller.

022. Configure a physical pin in Output Mode by modifying the mode register (MODER).

023. Set the output type to Push-Pull by manipulating the bits of the OTYPER register.

024. Configure a physical pin in Input Mode using the MODER register.

025. Enable the internal Pull-Up resistor on an input pin by writing to the PUPDR register.

026. Configure an input pin in internal Pull-Down mode using the PUPDR register.

027. Put a GPIO pin into Analog Mode to guarantee perfect electrical isolation and minimum power consumption.

028. Identify the appropriate Alternate Function of a serial peripheral by searching the chip's Datasheet.

029. Write to the AFRL (Alternate Function Low) register to assign an alternate function to a port pin (pins 0 to 7).

030. Write to the AFRH (Alternate Function High) register to assign an alternate function to pins 8 to 15.

031. Configure the switching speed (slew rate) of a pin by explicitly modifying the OSPEEDR register.

032. Activate an output pin by writing a logical '1' to the set bits of the BSRR atomic access register.

033. Deactivate an output pin by writing a logical '1' to the reset bits of the BSRR atomic access register.

034. Synchronously read the physical state of an input pin by inspecting the bits of the IDR register.

035. Design a software delay loop to implement a button debouncing routine.

LEVEL 3 — TIME MANAGEMENT AND INTERRUPTS (SYSTICK AND NVIC)

036. Locate the SysTick timer registers within the Cortex-M core memory space.
037. Configure the SysTick->LOAD register to set the exact number of cycles required for a time base.
038. Enable the SysTick timer without interrupts using the SysTick->CTRL register to perform polling.
039. Implement a delay function in milliseconds by monitoring the COUNTFLAG flag of the SysTick->CTRL register.
040. Modify the SysTick->CTRL register to enable the periodic interrupt of the core timer.
041. Write the SysTick service routine (SysTick_Handler) to increment a global millisecond counter.
042. Study the IPR (Interrupt Priority Registers) of the NVIC controller in the ARM Cortex core manual.
043. Manually configure the SCB's AIRCR register to define the priority grouping and subpriority splitting of the NVIC.
044. Write to the corresponding NVIC registers to assign a specific priority and subpriority to an interrupt vector.
045. Enable a peripheral interrupt in the NVIC by modifying the ISER (Interrupt Set-Enable Registers).
046. Configure the SYSCFG_EXTICR register to link a specific physical GPIO pin to an external interrupt line (EXTI).
047. Modify the EXTI_IMR register to unmask (enable) the desired external interrupt line.
048. Configure the EXTI_RTSR register to activate rising edge detection on the EXTI line.
049. Configure the EXTI_FTSR register to activate falling edge detection on the EXTI line.
050. Implement critical section protection by disabling global interrupts using the '__disable_irq' assembly instruction.
051. Restore global interrupts after a critical section using the '__enable_irq' assembly instruction.

052. Enable the DWT (Data Watchpoint and Trace) debug block by writing to the CoreDebug->DEMCR register.

053. Start the cycle counter of the DWT block by setting the initializer bit of the DWT->CTRL register to 1.

054. Measure the exact execution time of a block of code by subtracting the initial value from the final value of the DWT->CYCCNT register.

055. Design a synchronous main loop structure based on the millisecond flag generated by SysTick.

056. Configure two interrupts in the NVIC with different priority levels to verify hardware preemption.

057. Write the EXTIx_IRQHandler service routine corresponding to the configured line.

058. Read the interrupt flag in the EXTI_PR (Pending Register) to verify the source of the trigger.

059. Manually clear the pending bit in EXTI_PR by writing a logical '1' to it according to the hardware specification.

060. Clear the corresponding pending bit in the NVIC ICPR registers to conclude the interrupt cycle.

LEVEL 4 — HARDWARE TIMERS AND MODULATION (BASIC AND ADVANCED TIMERS)

061. Identify the full register map of the selected basic Timer (e.g., TIM6 or TIM2) in the Reference Manual.

062. Write the necessary value into the TIMx_PSC register to divide the clock frequency of the internal peripheral clock.

063. Calculate and write the exact auto-reload value into the TIMx_ARR register to define the overflow period.

064. Configure the CEN (Counter Enable) bit in the TIMx_CR1 register to start the Timer count.

065. Monitor the TIMx_CNT register via software to visually verify the synchronous increment of the count.

066. Modify the TIMx_DIER register to enable the update overflow interrupt (Update Interrupt Enable).

067. Write the TIMx_IRQHandler routine and clear the UIF flag in the TIMx_SR status register to prevent infinite loops.

068. Configure the bits of the TIMx_CCMRx register to set the timer channel to Output Compare Mode.

069. Configure PWM mode 1 or PWM mode 2 in the TIMx_CCMRx register for the corresponding Timer channel.

070. Write to the TIMx_CCER register to enable the Timer channel output to the assigned physical pin.

071. Write an initial value to the TIMx_CCRx capture compare register to establish the initial pulse width.

072. Implement a routine that dynamically modifies the value of TIMx_CCRx at runtime to alter the duty cycle.

073. Configure the TIMx_BDTR register to enable and calculate the deadtime between complementary outputs.

074. Configure the complementary synchronous outputs by activating the CCxE and CCxNE bits in the TIMx_CCER register.

075. Modify the TIMx_CCMRx register to set the channel to Input Capture Mode.

076. Configure the TIMx_CCER register to define whether input capture triggers on a rising edge, falling edge, or both.

077. Enable the input capture interrupt by activating the CCxIE bit in the TIMx_DIER register.

078. Read the value of the TIMx_CCRx register inside the ISR when the rising edge occurs and save it to memory.

079. Capture the second value in TIMx_CCRx on the falling edge and calculate the mathematical subtraction to obtain the pulse width.

080. Configure the SMS (Slave Mode Selection) bits in the TIMx_SMCR register to put the timer into Encoder Mode.

LEVEL 5 — SERIAL COMMUNICATION VIA REGISTERS (UART, SPI, I2C)

081. Locate the control registers of the USART/UART peripheral in the Reference Manual.

082. Configure the data word, stop bits, and parity by modifying the USART_CR1 and USART_CR2 registers.

083. Calculate the baud rate divisor value (USARTDIV) according to the APB bus clock frequency associated with the peripheral.

084. Write the calculated divisor value directly into the USART_BRR baud rate register.

085. Enable the transmission (TE) and reception (RE) blocks by modifying the USART_CR1 register.

086. Activate the UART peripheral globally by setting the UE (USART Enable) bit to 1 in USART_CR1.

087. Design a single-byte transmission function by implementing a polling loop that waits for the TXE flag in USART_SR.

088. Write the byte to be transmitted directly into the USART_TDR or USART_DR data register.

089. Design a single-byte reception function via polling by waiting for the RXNE flag in the USART_SR register to activate.

090. Read the received byte by accessing the USART_RDR or USART_DR data register immediately.

091. Enable the UART reception interrupt by activating the RXNEIE bit in the USART_CR1 register.

092. Write the UARTx_IRQHandler service routine to intercept incoming data asynchronously.

093. Map and configure the control registers of the SPI peripheral (SPI_CR1 and SPI_CR2).

094. Configure the data word size (8 or 16 bits) and Master mode in the SPI_CR1 register.

095. Configure the CPOL and CPHA bits in SPI_CR1 to define the SPI clock polarity and phase according to the slave.

096. Configure the baud rate prescaler (BR) bits in SPI_CR1 to adjust the SPI clock speed.

097. Implement manual control (Software Slave Management) by setting the SSM and SSI bits to 1 in the SPI_CR1 register.

098. Design functions to drive the GPIO pin assigned as Chip Select (CS) low and high via software.

099. Develop the SPI transmission/reception routine via polling by monitoring the TXE and RXNE flags in SPI_SR.

100. Write and implement the configuration of an SPI peripheral in Slave Mode by explicitly modifying the SPI_CR1 register.

101. Configure the I2C_CR1, I2C_CR2, and timing (I2C_TIMINGR) registers following the manual's formulas.

102. Program the automatic generation of the START condition by writing to the corresponding bits of the I2C_CR2 register.

103. Write the 7-bit slave device address in the SADD field of the I2C_CR2 register.

104. Configure the number of bytes to transmit in the NBYTES field of the I2C_CR2 register.

105. Monitor the TXIS (Transmit Interrupt Status) flag before writing each byte into the I2C_TXDR transmission register.

106. Verify reception by analyzing the I2C RXNE flag and read data from the I2C_RXDR register.

107. Monitor and intercept the TC (Transfer Complete) flag or the NACKF flag in case of communication failure.

108. Generate the STOP condition manually or automatically by programming the STOP bit in I2C_CR2.

109. Design a loop that sequentially traverses the 127 possible addresses of the I2C bus by sending a test frame.

110. Analyze the ACK or NACK flag returned by the bus to identify and record addresses with live devices.

LEVEL 6 — DATA STRUCTURES AND MEMORY OPTIMIZATION (THE RING BUFFER)

111. Define a 'struct' in C representing a Ring Buffer, containing the array, head, tail, and size.
112. Implement the Ring Buffer initialization function by explicitly setting the head and tail indices to zero.
113. Develop the Ring Buffer write function, mathematically evaluating if the buffer is full before inserting.
114. Develop the Ring Buffer read function, evaluating if head equals tail to handle the empty condition.
115. Integrate the Ring Buffer write function inside the UARTx_IRQHandler reception interrupt routine.
116. Design a secondary linear buffer dedicated exclusively to packaging complete data frames before processing.
117. Study the syntax of the GCC compiler section attribute ('__attribute__((section(".section_name")))').
118. Apply the section attribute to a global array to force its placement in a specific zone of SRAM memory.
119. Open and analyze the linker configuration file (Linker Script with '.ld' extension) of your project.
120. Modify the MEMORY section of the Linker Script to declare a new physical RAM memory region.
121. Define a custom output section in the Linker Script to route specific variables toward the new region.
122. Apply the manual alignment directive to data structures to force blocks aligned to full 32-bit words.
123. Declare a constant array initialized with static values by prefixing the 'const' qualifier in C.
124. Verify in the compilation map file ('.map' file) that the 'const' array is stored exclusively in Flash memory.

125. Implement an algorithm at code startup that fills the entire Stack with a fixed binary pattern ("canary").
126. Design a diagnostic function that reads the stack memory live to count how many "canaries" are still alive.
127. Calculate and report the Stack watermark at runtime to monitor real RAM consumption.

LEVEL 7 — AUTOMATION AND DIRECT TRANSFER (DMA CONTROLLERS)

128. Locate the configuration register addresses of the DMA controller (DMA1 or DMA2) in the reference manual.
129. Identify and map the DMA_CPARx (peripheral address), DMA_CMARx (memory address), and DMA_CNDTRx (number of data items) registers.
130. Configure the source block address and destination block address in the CPARx and CMARx registers for a Memory-to-Memory flow.
131. Write the exact transfer size into the CNDTRx decremental data counter register.
132. Configure automatic address increment for both source and destination in the DMA_CCRx channel control register.
133. Activate the EN (Enable) bit in the DMA_CCRx register to trigger Memory-to-Memory block transfers immediately.
134. Configure a DMA channel by assigning the physical address of the USART_RDR register into the CPARx register.
135. Assign the base address of an SRAM array inside the CMARx memory register of the DMA channel.
136. Enable the DMA transfer request on the peripheral side by activating the DMAR bit in the UART_CR3 register.
137. Modify the channel control register of the DMA to enable the Transfer Complete Interrupt (TCIE).
138. Enable the DMA channel interrupt vector in the NVIC and implement the DMAx_Channelx_IRQHandler service routine.
139. Evaluate the TC (Transfer Complete) flag inside the DMA ISR to activate processing of the received data frame.
140. Explicitly clear the TC flag by writing to the DMA flag clear register (DMA_IFCR).
141. Configure the DMA channel to enable the Half-Transfer Interrupt by activating the HTIE bit in the DMA_CCRx register.
142. Implement a double-buffer (Ping-Pong) technique using the DMA ISR to process one half of the buffer while the DMA fills the other.

143. Configure the CIRC (Circular Mode) bit in the DMA channel control register to automate continuous acquisition.
144. Map and study the configuration registers of the DMA request multiplexer (DMAMUX).
145. Configure the DMAMUX register to link a Timer update trigger with the execution request of the DMA channel.
146. Configure the ADC channels to operate in sequential multi-channel scan mode.
147. Enable the DMA request within the ADC control registers (DMA or DMACFG bit).
148. Connect the DMA channel to the ADC data register (ADC_DR) to transfer readings directly to memory without CPU intervention.
149. Configure a DMA channel to point to the SPI transmit register (SPI_DR) and enable the TXDMA bit in SPI_CR2.
150. Initiate automated massive data burst transfers to displays or sensors using SPI backed by DMA.

LEVEL 8 — CORE ARCHITECTURE AND FAULT DEBUGGING (CORTEX-M CORE)

151. Study GCC inline assembly syntax ('__asm volatile').
152. Write inline assembly code to read internal register R0 of the ARM processor and dump its value into a C variable.
153. Write assembly instructions to directly extract the LR (Link Register) and PC (Program Counter) special registers.
154. Configure the GCC compiler to export the complete disassembly file (file with '.list' or '.disasm' extension).
155. Audit the disassembly file by analyzing the actual instructions generated by the compiler for the main loop.
156. Locate the HardFault exception vector in the vector table and implement the 'HardFault_Handler' function.
157. Write an assembly block inside 'HardFault_Handler' to detect whether the processor was using the Main Stack or the Process Stack.
158. Extract the current stack pointer (MSP or PSP) and pass it as an argument to a diagnostic function in C.
159. Decode the stack-backed register block to extract the exact PC value at the moment of system collapse.
160. Map and study the configuration registers of the MPU (Memory Protection Unit) block of the Cortex-M core.
161. Configure an MPU region defining its base address, size, and access permissions using the MPU_RBAR and MPU_RASR registers.
162. Establish a memory zone as Non-Executable (XN bit - Execute Never) using the MPU to mitigate code injection.
163. Enable system exceptions for memory faults by modifying the SHCSR register of the SCB block.
164. Intentionally provoke a UsageFault by executing a division by zero after activating the corresponding bit in SCB->CCR.
165. Read the UFSR (UsageFault Status Register) inside its handler to verify the exact cause of the fault.

166. Modify the CONTROL register of the ARM core using assembly instructions to toggle the processor privilege level.
167. Develop a safe transition from Privileged Mode to Unprivileged Mode to isolate secondary code execution.
168. Map the control registers of the IWDG (Independent Watchdog) peripheral.
169. Write the unlock key into the IWDG_KR register to allow editing of the prescaler and reload registers.
170. Configure the Watchdog overflow time and start the count by writing the boot command to IWDG_KR.
171. Insert the Watchdog refresh instruction (feeding the dog) strategically inside the system's main loop.
172. Locate CMSIS architecture functions for data cache control (SCB_EnableDCache, SCB_CleanDCache).
173. Implement cache clean routines (Clean) before the DMA reads data modified by the CPU in high-performance microcontrollers.
174. Implement cache invalidation routines (Invalidate) before the CPU reads RAM zones modified by the DMA.

LEVEL 9 — ADVANCED ARCHITECTURES AND MULTICORE (STM32H7 DUAL-CORE)

175. Locate the HSEM (Hardware Semaphore) register block in the manual, specific to multicore synchronization.
176. Implement a routine that attempts to take a hardware semaphore by writing the core ID to the HSEM_Rx register.
177. Read the HSEM status register to synchronously verify if the lock was successfully assigned to the current core.
178. Design the hardware semaphore release logic by writing the unlock key to the HSEM_R_x register.
179. Analyze the boot sequence of the dual-core architecture (e.g., Cortex-M7 and Cortex-M4 in the H755 line).
180. Program the RCC register configuration to hold the second core (Cortex-M4) in its initial reset state.
181. Write code on the main core (Cortex-M7) that safely configures global system clocks.
182. Release the reset state of the second core by modifying the corresponding control bits in the RCC from Core 1.
183. Map a specific region of shared SRAM memory (SRAM4 / AXI SRAM) defined in the Linker Script for both cores.
184. Design a structured data structure within the shared SRAM region to act as an inter-core messaging channel.
185. Study inter-processor communication (IPCC) registers or cross software interrupts.
186. Configure the software interrupt generation register to send an asynchronous notification pulse to the sibling core.
187. Implement the inter-core interrupt service routine to immediately read and empty the shared buffer upon receiving the pulse.
188. Study ARM assembly instructions specialized in atomic synchronization: LDREX (Load Exclusive) and STREX (Store Exclusive).
189. Implement a block of code that attempts to read a shared control variable using the LDREX assembly instruction.

190. Execute the conditional atomic write using the STREX instruction, evaluating the status returned by the processor.
191. Design a software mutual exclusion mechanism free of race conditions based entirely on LDREX/STREX.
192. Modify the SCR (System Control Register) of the SCB block to configure the processor's SLEEPDEEP bit.
193. Configure PWR block registers to define the desired low-power level (e.g., Stop or Standby Mode).
194. Execute the WFI (Wait For Interrupt) assembly instruction to induce the microcontroller into a dormant low-power state.
195. Develop a routine that selectively disables the clocks of all inert peripherals in the RCC to reduce dynamic consumption.
196. Analyze the topology and internal bus matrix of the chip, identifying potential bottlenecks due to synchronous access collisions.
197. Design buffer distribution by assigning the DMA buffer to memory accessible by its bus without colliding with the CPU stack.
198. Design a basic application structured as a Bootloader located in the first sectors of the microcontroller's Flash memory.
199. Implement the jump routine by modifying the main stack pointer (MSP) with the base address of the target application.
200. Load the Program Counter (PC) with the address of the new application's reset vector to force execution.
201. Insert memory barrier directives in assembly ('__DSB()' Data Synchronization Barrier and '__ISB()' Instruction Synchronization Barrier) to guarantee strict memory access ordering before jumping or switching contexts.

LEVEL 10 — REAL-TIME OPERATING SYSTEMS (RTOS AND FORMAL SCHEDULING)

202. Configure the 'FreeRTOSConfig.h' header file, adjusting the system clock frequency and Kernel tick rate.
203. Map critical processor interrupt vectors by assigning SVC_Handler, PendSV_Handler, and SysTick_Handler to the RTOS Kernel.
204. Declare task handlers ('TaskHandle_t') and write concurrent thread functions with isolated infinite loops.
205. Invoke the 'xTaskCreate' function to instantiate multiple tasks, explicitly assigning static priority levels.
206. Configure RTOS debugging directives to activate active monitoring of the individual Stack assigned to each task.
207. Capture runtime failures by implementing the stack overflow hook function ('vApplicationStackOverflowHook').
208. Instantiate a structured message queue by invoking the Kernel initialization function 'xQueueCreate'.
209. Develop a producer task that packages data and inserts it into the message queue using the 'xQueueSend' function.
210. Develop a consumer task that extracts and processes data, blocking efficiently via 'xQueueReceive'.
211. Instantiate a system-invoked binary semaphore by calling the 'xSemaphoreCreateBinary' creation function.
212. Configure a hardware interrupt routine (peripheral ISR) that unlocks a high-priority task by releasing the semaphore.
213. Strictly use functions with the special suffix for RTOS interrupts ('xSemaphoreGiveFromISR') inside the ISR.
214. Invoke the 'portYIELD_FROM_ISR' directive at the end of the ISR to force an immediate context switch if the unlocked task has priority.
215. Create a simulated shared resource in memory and protect its access by instantiating a Mutex using the 'xSemaphoreCreateMutex' function.

216. Implement taking and releasing the Mutex ('xSemaphoreTake' / 'xSemaphoreGive') surrounding the critical code block in each task.
217. Deliberately design a priority inversion scenario by configuring three tasks (High, Medium, Low) and a common Mutex.
218. Monitor the unexpected stall of the High-priority task when indirectly blocked by the Medium-priority task.
219. Enable the native priority inheritance feature of the Mutex in the RTOS to validate how the system automatically mitigates inversion.
220. Instantiate an event group ('xEventGroupCreate') and implement multi-phase synchronization where a task waits for multiple simultaneous binary flags.
221. Configure an RTOS software timer ('xTimerCreate') associating a callback function that executes upon expiration.
222. Gather worst-case execution times (WCET) and strict execution periods for all designed concurrent tasks.
223. Manually apply the Rate Monotonic Scheduling (RMS) mathematical formula over the set of tasks.
224. Calculate the theoretical CPU utilization limit to formally verify if the embedded system guarantees compliance with all real-time deadlines without dropouts.